



Analytics Engineer

Docker



Sommaire

Docker

1. Pourquoi Docker
2. Définitions clés
3. Les concepts clés
 - 3.1. Construire une image
 - 3.2. Concepts avancés
4. Cas d'usage en AE



1. Pourquoi Docker

Docker résout les problèmes de compatibilité entre environnements (macOS, Linux, Windows).

Il permet de :

- Standardiser les environnements d'exécution
- Éviter les erreurs liées aux versions de bibliothèques ou systèmes
- Gagner en portabilité, reproductibilité et collaboration

2. Définitions clés

- **Image** : modèle figé d'environnement, prêt à être exécuté
- **Container** : instance active d'une image (comme un plat réchauffé)
- **Dockerfile** : fichier d'instructions pour construire une image
- **Docker CLI** : ensemble de commandes pour interagir avec les images et containers

3. Concepts Clés

3.1 Construire une image

On utilise un Dockerfile pour définir :

- L'OS de base (ex. Ubuntu, Alpine)
- Les dépendances
- Le code à copier
- La commande à exécuter au démarrage
- Sources d'images : Docker Hub (images officielles et communautaires)



3.2 Concepts avancés

Entrypoint vs. CMD

- **ENTRYPOINT** : commande principale (non modifiable)
- **CMD** : arguments par défaut (modifiables au run)

Volumes

Permettent de persister les données ou de partager des fichiers :

- **Bind mounts** : `-v /host/path:/container/path`
- **Named volumes** : `-v nom_volume:/container/path`

4. Cas d'usage en AE

Un container pour Airflow et / ou un container pour dbt facilite l'orchestration et la reproductibilité du projet.



Airbyte

